

tation. This was done by Microsoft to accelerate deep learning.

Why general-purpose processors are falling behind

General-purpose processors are powerful. You can implement any instructions you want. There is an instruction called the exclusion pipeline. If you just want to add an operation, then you will have to fetch the instruction and decode it to know which instruction it is.

Sometimes, the data may not be available if you want to rename one of the operands or both operands in order to avoid the name conflict, for which you will have to schedule in a renovation station. You can execute all the data in the arithmetic logic units (ALUs) once all the data is available.

If you are doing an out of order execution, then you will have to have a reorder buffer so that everyone before you is committed and you can write it back.

Power of customisation (domain-specific accelerators)

Following are some of the powers of customisation:

- Special data types and operations. Get things done in just one cycle what normally takes tens or hundreds of cycles.
- Massive parallelism. This increases the performance but not the efficiency.
- Optimised memory. It provides high bandwidth (consumes low energy) for specific data structures and operations.
- Reduced or amortised overhead. Provides 10,000 times efficiency gains for simple operations.
- Algorithm-architecture co-design can also be done.

Now, this raises the question, why everyone is not designing their own customised accelerators? The reason is that most people in the IT industry are software developers.

• Example: 1 layer of the CNN algorithm

```
// Skip const variable initialization and macro definitions due to page limit

void CnnKernel(const float* input, const float* weight,
               const float* bias, float* output){
    float C[ParallelOut][InSize][InSize];
    for (int i = 0; i < NumOut / ParallelOut; ++i) {
        // Skip initialization due to page limit
        // Convolution
        for (int j = 0; j < NumIn; ++j) {
            for (int h = 0; h < InSize; ++h) {
                for (int w = 0; w < InSize; ++w) {
                    for (int po = 0; po < ParallelOut; po++) {
                        for (int p = 0; p < kKernel; ++p) {
                            for (int q = 0; q < kKernel; ++q) {
                                C[po][h][w] += weight(i, po, j, p, q) *
                                input(j, h + p, w + q); } } } }
                // Skip pooling due to page limit
            }
        }
    }
}
```

1 layer of the CNN algorithm

There are only 66,000 persons who are considered computer power engineers, which makes the FPGAs (and ASICs) designing quite challenging.

What about high-level synthesis (HLS)?

HLS tools allow you to do a C based synthesis. HLS supports standard floating point operations and it consists of a lot of special optimisations. It fully supports IEEE-754 floating point data types and operations. It can efficiently handle bit-accurate fixed-point arithmetic. The quality of result matches or exceeds manual RTL for many designs.

However, not all C programs lead to good performance. Let us take a very simple example like a convolutional neural network (CNN). Basically, for every output image you will have to do this convolution. It has to be done for every row and column. You just have to feed this code (see figure) into that high-level synthesis tool.

You do get a circuit, but it is 80 times slower than even a single-core central processing unit (CPU). You

just have to take the parameters (const float* input, const float* weight, const float* bias, float* output) and map them into the device DRAM.

When you design your own accelerator, you have to get the cache from D1 every time, unless you have specified. If you don't do any parallelisation, then all these loops will be executed sequentially. It is important to remember that in FPGA the clock frequency is typically five times slower.

What it takes to get a good HLS design? Since the speed is lower than a single-core CPU, students are not satisfied with the results, hence, they optimise it. They introduce on-chip buffers to cache the data, and they do fine-grain pipelining, fine-grain parallelisation, and coarse grain pipelining so that computation can be overlapped with communication. At the end, memory partition is done in order to maintain concurrency.

Making customised computing accessible to everyone

We want a software programmer to start with the language they are